Type Classes via Dictionaries

asses · JSON encoder + Show/Eq for domain records and composites · Build with

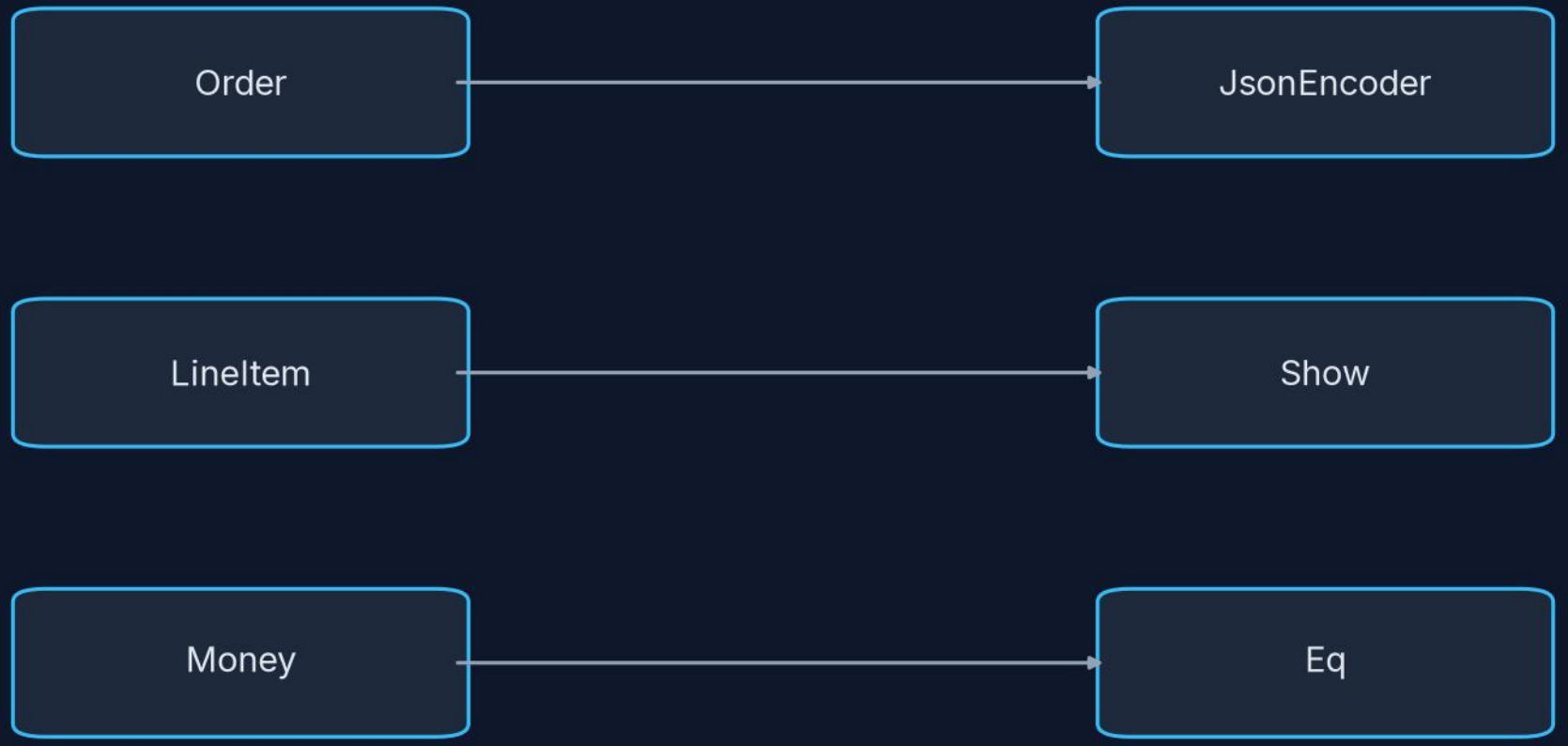
Today's Hook

- Problem: JSON encoder + Show/Eq for domain records and composites
- Pattern: Type Classes — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- JSON encoder + Show/Eq for domain records and composites
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Type Classes: JSON encoder + Show/Eq for domain records and comp

Real-World Scenario

- Running example: JSON encoder + Show/Eq for domain records and composites
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.typeclass.problem;

/** Naive: ad-hoc toString/JSON in every call site — no reuse, easy to drift. */
public final class NaiveJson {
    public static String moneyJson(TypeClasses.Money m) {
        return "{cents:" + m.cents() + "}"; // forgot currency
    }
    public static String orderJson(TypeClasses.Order o) {
        return "{id:" + o.id() + "}"; // forgot items
    }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.typeclass.problem;

import java.util.List;
import java.util.Objects;
import java.util.stream.Collectors;

/** Explicit type-class dictionaries: Show, Eq, JsonEncoder for domain records. */
public final class TypeClasses {
    private TypeClasses() {}

    public record Money(String currency, int cents) {}
    public record LineItem(String sku, int qty, Money price) {}
    public record Order(String id, List<LineItem> items) {}

    public interface Show<A> { String show(A a); }
    public interface Eq<A> { boolean eqv(A a, A b); }
    public interface JsonEncoder<A> { String encode(A a); }

    public static final Show<Money> showMoney = m -> m.currency() + " " + (m.cents() / 100.0);
    public static final Eq<Money> eqMoney = (a, b) ->
            Objects.equals(a.currency(), b.currency()) && a.cents() == b.cents();
    public static final JsonEncoder<Money> jsonMoney = m ->
            "{\"currency\":\"" + m.currency() + "\",\"cents\":" + m.cents() + "}";
```

Solution (2)

```
    public static final Show<LineItem> showLine = li ->
            li.qty() + "x" + li.sku() + "@" + showMoney.show(li.price());
    public static final JsonEncoder<LineItem> jsonLine = li ->
            "{\"sku\":\"" + li.sku() + "\",\"qty\":" + li.qty() + ",\"price\":" + jsonMoney.encode(li.price()) + "}";

    /** Derivation for composites. */
    public static final JsonEncoder<Order> jsonOrder = o -> {
        String items = o.items().stream().map(jsonLine::encode).collect(Collectors.joining(","));
        return "{\"id\":\"" + o.id() + "\",\"items\":[" + items + "]}";
    };
    public static final Show<Order> showOrder = o ->
            "Order(" + o.id() + ": " + o.items().stream().map(showLine::show).collect(Collectors.joining(", ")) + ")";
    public static final Eq<Order> eqOrder = (a, b) ->
            Objects.equals(a.id(), b.id()) && a.items().equals(b.items());
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.typeclass.problem;

import java.util.List;

public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Type Classes: Show / Eq / JsonEncoder ===");
        var m = new TypeClasses.Money("USD", 1999);
        var order = new TypeClasses.Order("o1", List.of(
                new TypeClasses.LineItem("SKU1", 2, m),
                new TypeClasses.LineItem("SKU2", 1, new TypeClasses.Money("USD", 500))));
        System.out.println("--- naive (incomplete JSON) ---");
        System.out.println(NaiveJson.moneyJson(m));
        System.out.println(NaiveJson.orderJson(order));
        System.out.println("--- type-class dictionaries ---");
        System.out.println("show=" + TypeClasses.showOrder.show(order));
        System.out.println("json=" + TypeClasses.jsonOrder.encode(order));
        System.out.println("eqSelf=" + TypeClasses.eqOrder.eqv(order, order));
        System.out.println("eqOther=" + TypeClasses.eqOrder.eqv(order, new TypeClasses.Order("o2", List.of())));
    }
}
```

Walkthrough Output

```
=== Type Classes: Show / Eq / JsonEncoder ===
--- naive (incomplete JSON) ---
{cents:1999}
{id:o1}
--- type-class dictionaries ---
show=Order(o1: 2xSKU1@USD 19.99, 1xSKU2@USD 5.0)
json={"id":"o1","items":[{"sku":"SKU1","qty":2,"price":{"currency":"USD","cents":1999}},{"sku":"SKU2","qty":1,"price":{"currency":"USD","cents":500}}]}
eqSelf=true
eqOther=false
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Type Classes.

Related Patterns

- Contrast Type Classes with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Type Classes → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.